

R8 — Interrupções e repique

O hardware chama o programa, e o botão mente sobre quantas vezes foi apertado

Microcontroladores — DENE/UFMT

OBJETIVOS

- Fazer o hardware chamar o programa: uma base de tempo de 5 ms por interrupção, medida no osciloscópio.
- Medir o repique de um botão real, e contar o que ele faz com uma interrupção de borda.
- Implementar o filtro por confirmação da aula 7, e medir o que ele custa em atraso.
- Mostrar que um sinalizador guarda “aconteceu”, e não “quantas vezes”.
- Tocar o lá de 440 Hz por interrupção enquanto o display é atualizado.

CH2-1 (LCD)	ON — mostra as contagens	NESTE KIT
CH5 (relés)	Todas em OFF — não há relé neste curso de bancada	
Chaves SWITCHS (PORTB)	OFF	
Botões	INT0 (SW12, RB0) é o botão medido; INT1 (SW13, RB1) é o segundo, só para o osciloscópio	
CH3-6 (BUZZER)	OFF até a Tarefa 6; então ON	
CH3-2 a CH3-5 e CH3-7	OFF	

A verificar antes da sessão: o deslocamento de vetor que o bootloader do XM118 usa, e se o projeto de exemplo do curso já traz o arquivo de ligação correspondente (aula 7). Se a interrupção nunca executar, é o primeiro suspeito. E em qual conector está RE2, o pino que marca o tique.

Pontuação. As seis tarefas somam 8,0, e as previsões P1 a P5 da aula 7, preenchidas antes da sessão, valem 2,0. A nota de cada tarefa e de cada previsão está na margem, ao lado dela. A seção *Se sobrar tempo*, no fim, não vale nota: é para quem terminar antes.

Osciloscópio: ponta em 10×, acoplamento DC, disparo **único** (single) na borda de descida do canal 1 para capturar o repique; base de tempo de 1 a 2 ms/div.

BANCADA

1 Previsões — entregues no início da sessão

As mesmas perguntas do fim da aula 7. Quem trouxe a folha preenchida copia aqui as respostas; a nota é do raciocínio, não do número.

P1. Capturando um toque no osciloscópio, quantos milissegundos de repique você espera ver? E quantas transições?

PREVISÃO · 0,4 pt

P2. Contando eventos por toque sem filtro, quantos espera? Tocando dez vezes, o número se repete?

PREVISÃO · 0,4 pt

P3. Com amostragem a 5 ms e confirmação de 3, qual é o atraso entre encostar no botão e o programa reagir? É perceptível?

PREVISÃO · 0,4 pt

P4. Com confirmação de 10 amostras, o que melhora e o que piora? Dê os dois números.

PREVISÃO · 0,4 pt

P5. O repique de dois botões diferentes da placa é o mesmo?

PREVISÃO · 0,4 pt

2 O programa

Um programa só, para a sessão inteira. Três fontes de interrupção, um tratamento:

- **INT0**, na borda de descida de RB0: conta **toda** borda, sem filtro (B).
- **Timer0**, a cada 5 ms: inverte RE2, para medir o tique, e aplica o filtro por confirmação da aula 7 ao mesmo botão (F).
- **Timer1**, a cada meio período de 440 Hz: inverte RC2, o buzzer. Só é ligado na Tarefa 6.

O laço principal consome o sinalizador borda e conta os apertos que ele viu (A). O display mostra os três números.

```
#define _XTAL_FREQ 16000000UL
#include <xc.h>
#include <stdint.h>
#include "lcd.h"

#define PRECARGA_5MS 45536u          /* 65536 - 20000: 5 ms a 250 ns */
#define PRECARGA_LA 60991u          /* 65536 - 4545: meio periodo de 440 Hz */
#define CONFIRMA 3u                  /* amostras iguais para aceitar (T4) */
#define BUZZER 0                     /* 1 na Tarefa 6 */
/* #define TAREFA() __delay_ms(300) -- Tarefa 5 */

volatile uint16_t brutos = 0;        /* bordas vistas pelo INT0, sem filtro */
volatile uint16_t filtrados = 0;     /* apertos confirmados pelo filtro */
volatile uint8_t borda = 0;          /* sinalizador: "houve um aperto" */

void __interrupt() tratar(void)
{
    static uint8_t candidato = 1, iguais = 0, botao = 1; /* so o tratamento ve */

    if (INTCONbits.INT0IF) {          /* borda de descida em RB0 */
        INTCONbits.INT0IF = 0;
        brutos++;
    }

    if (INTCONbits.TMR0IF) {           /* passaram 5 ms */
        INTCONbits.TMR0IF = 0;
        TMR0H = (uint8_t)(PRECARGA_5MS >> 8);
        TMR0L = (uint8_t)(PRECARGA_5MS & 0xFF);
        LATEbits.LATE2 ^= 1;          /* marca o tique (T1) */

        uint8_t agora = PORTBbits.RB0;
        if (agora != candidato) {
            candidato = agora;
            iguais = 1;
        } else if (iguais < CONFIRMA) {
            iguais++;
            if (iguais == CONFIRMA && candidato != botao) {
                botao = candidato;
                if (botao == 0u) {      /* pressionado, confirmado */
                    filtrados++;
                    borda = 1;
                }
            }
        }
    }
}
```

```

    if (PIR1bits.TMR1IF) {                                /* meio periodo do la (T6) */
        PIR1bits.TMR1IF = 0;
        TMR1H = (uint8_t)(PRECARGA_LA >> 8);
        TMR1L = (uint8_t)(PRECARGA_LA & 0xFF);
        LATCbits.LATC2 ^= 1;
    }
}

/* Leitura de 16 bits que o tratamento escreve: le duas vezes ate concordar. */
static uint16_t ler16(volatile uint16_t *p)
{
    uint16_t a, b;
    do {
        a = *p;
        b = *p;
    } while (a != b);
    return a;
}

void main(void)
{
    uint16_t aceitos = 0;                                /* apertos que o laço consumiu */

    ADCON1 = 0x0F;                                        /* RB0 (INT0) e RE0-RE2 digitais */
    TRISBbits.TRISB0 = 1;
    LATEbits.LATE2 = 0;  TRISEbits.TRISE2 = 0;
    LATCbits.LATC2 = 0;  TRISCbits.TRISC2 = 0;
    lcd_iniciar();

    TMR0H = (uint8_t)(PRECARGA_5MS >> 8);  TMR0L = (uint8_t)(PRECARGA_5MS & 0xFF);
    T0CON = 0x88;                                /* 16 bits, interno, sem divisor */
    INTCONbits.TMR0IF = 0;  INTCONbits.TMR0IE = 1;

    INTCON2bits.INTEDG0 = 0;                    /* INT0 na borda de descida: ao apertar */
    INTCONbits.INT0IF = 0;  INTCONbits.INT0IE = 1;

    TMR1H = (uint8_t)(PRECARGA_LA >> 8);  TMR1L = (uint8_t)(PRECARGA_LA & 0xFF);
    T1CON = 0x81;                                /* 16 bits, 1:1, interno, ligado */
    PIR1bits.TMR1IF = 0;  PIE1bits.TMR1IE = BUZZER;

    INTCONbits.PEIE = 1;                        /* o Timer1 e periferico */
    INTCONbits.GIE = 1;

    for (;;) {
        if (borda) {                                /* trabalho diferido: le e zera */
            borda = 0;
            aceitos++;
        }

        lcd_posicao(0, 0);
        lcd_texto("B=");  lcd_numero(ler16(&brutos), 5);
        lcd_texto(" F=");  lcd_numero(ler16(&filtrados), 5);
        lcd_posicao(1, 0);
        lcd_texto("A=");  lcd_numero(aceitos, 5);

        /* TAREFA(); */
    }
}

```

O tratamento lê um pino, mexe em algumas variáveis e sai — nada de display, nada de divisão. candidato, iguais e botao são static e não volatile: só o tratamento os enxerga. brutos, filtrados e borda atravessam a fronteira, e por isso são volatile; os dois de 16 bits são lidos por ler16, porque volatile não os torna indivisíveis.

NOTA

3 Parte 1 — o hardware chama o programa

Tarefa 1. Grave o programa e meça RE2 no osciloscópio.

TAREFA · 1,0 pt

	Previsto	Medido
Período da onda em RE2		
Frequência		

(a) Nenhuma linha do laço chama tratar. Quem chama, e em que instante?

(b) No R6, o período em RD0 dependia da tarefa do laço. Aqui o laço escreve no display o tempo todo. O período mudou? Por quê?

4 Parte 2 — o repique, medido e contado

Tarefa 2. Canal 1 no pino do botão INT0 (RB0), disparo único na borda de descida. Aperte uma vez e congele a captura. Repita três vezes, e depois faça o mesmo com o botão INT1 (RB1).

TAREFA · 1,5 pt

	Toque 1	Toque 2	Toque 3
INT0: duração do repique			
INT0: número de transições			
INT1: duração do repique			

(a) Confere com a P1? E com a P5 — os dois botões repicam igual?

(b) Com base na maior duração medida, 5 ms de amostragem e 3 confirmações são suficientes? Justifique com os números.

Tarefa 3. Zere o kit. Aperte o botão INT0 dez vezes, devagar, e registre B e F. Três rodadas, zerando entre elas.

TAREFA · 1,5 pt

	Rodada 1	Rodada 2	Rodada 3
B (INT0, sem filtro)			
F (filtro, 3×5 ms)			

(a) Confere com a P2? Por que B passa de dez e varia?

(b) O INT0 e o filtro leem o **mesmo** pino, no mesmo programa. Por que um conta errado e o outro certo?

Uma entrada de interrupção por borda é a pior forma de ler um botão mecânico: ela é rápida e fiel, e por isso conta o repique. A interrupção certa para um botão é a do **temporizador**, que amostra o pino num ritmo que o repique não acompanha.

CONCEITO

5 Parte 3 — o custo do filtro, e o limite do sinalizador

Tarefa 4. Troque CONFIRMA para 10 e grave. Faça dez toques normais, e depois dez toques o mais **rápidos** que conseguir.

TAREFA · 1,0 pt

	CONFIRMA = 3	CONFIRMA = 10
F com toques normais		
F com toques rápidos		

Confere com a P3 e a P4? Qual é o atraso de confirmação em cada caso?

Tarefa 5. Volte CONFIRMA a 3. Descomente as duas linhas de TAREFA(): o laço passa a demorar 300 ms por volta. Faça dez toques rápidos e registre F e A.

TAREFA · 1,0 pt

F (confirmados pelo tratamento)
A (consumidos pelo laço)

(a) F e A deram iguais? Qual perdeu, e onde?

(b) Proponha a correção, em uma linha no tratamento e uma no laço.

Um uint8_t como sinalizador guarda “aconteceu”, não “aconteceu duas vezes”. O tratamento curto e o trabalho diferido continuam certos; o que a Tarefa 5 mostra é que o canal entre os dois precisa ter a capacidade do que passa por ele — um bit, um contador, ou uma fila.

CONCEITO

6 Parte 4 — a nota que não depende da tela

Tarefa 6. Troque BUZZER para 1, ligue CH3-6 e grave (com TAREFA() ainda descomentada). O Timer1 inverte RC2 a cada 4545 ciclos, pela interrupção.

TAREFA · 1,0 pt

Frequência medida em RC2: _____

(a) A nota falha enquanto o display é atualizado, ou durante os 300 ms da tarefa? Por quê?

(b) A frequência medida fica um pouco abaixo de 440 Hz. De onde vem a diferença?

7 Armadilhas frequentes

Sintoma	Causa provável
O tratamento nunca executa	Vetor da aplicação não bate com o do bootloader; ou GIE em 0
O Timer1 não interrompe	Faltou PEIE = 1: ele é periférico
Programa “trava” logo ao ligar	Um indicador não foi zerado no tratamento: ele volta a ser chamado para sempre
B não conta	INT0IE em 0, ou ADCON1 não é 0x0F e RB0 (AN12) está analógico
Funciona em depuração, trava no normal	Variável compartilhada sem volatile
Número absurdo, raro, no display	Leitura de 16 bits sem ler16: os dois bytes vieram de valores diferentes

8 Entrega

As tabelas das Tarefas 1 a 6 e as três capturas de repique da Tarefa 2.

TAREFA

Responda também:

(a) Divida o programa em “no tratamento” e “no laço”. Por que o display e a divisão ficaram do lado do laço?

(b) O R6 mediu a deriva da recarga por consulta. Qual das tarefas de hoje mostra que ela sumiu, e qual mostra que ela não sumiu por completo?

9 Se sobrar tempo

E1 — o volatile, na prática. Tire o volatile de borda, compile com otimização e grave. O laço ainda vê os apertos?

OPCIONAL

E2 — o filtro no simulador. Com a captura da Tarefa 2 convertida em amostras de 5 ms (1 = solto, 0 = apertado), rode a função de filtro no PICSIMLab ou em Python e confira quantos apertos ela aceita. O simulador não tem repique; a captura dá a ele o repique da sua bancada.

OPCIONAL

E3 — o lá afinado. Desconte da pré-carga os ciclos entre o estouro e a escrita de TMR1 (medidos pela diferença entre 440 Hz e a frequência da Tarefa 6) e meça de novo.

OPCIONAL

No R9: o estágio de potência, e a pergunta de por que o processador reiniciou. No R11, a serial transmite pela mesma estrutura de hoje — o tratamento entrega um byte, o laço prepara a linha.

NOTA